



Computer Graphics
Spring Semester 2025

Group members
Sarah Algarni
Shatha Alshaikh
Shayma Aljuaid
Lujain Alqarni
Ohoud Alzahrani
Lina ALSihman

Table of Contents

1	Introduction.....	3
2	Executive Summary.....	3
3	Abstraction.....	5
4	Algorithms and Computational Implementation.....	7
4.1	Pseudocode and C# Scripts for Game Scripts and Systems.....	7
4.1.1	Star Reward System.....	7
4.1.2	Clearing the Land Scene.....	11
4.1.3	Planting the Seeds Scene.....	16
4.1.4	Watering the Tree Scene.....	25
4.2	Dialogue Code.....	30
4.3	OpenGL Code Explanation.....	34
4.3.1	OpenGL Result.....	35
4.3.2	OpenGL Code	36
5	Results.....	40
5.1	3D Objects	40
5.2	Buttons	45
5.3	Shots from the Game.....	46
6	Conclusion	54
6.1	Advantages of the Approach.....	54
6.2	Disadvantages of the Approach.....	54
6.3	Unexpected Aspects.....	54
6.4	Future Work	55

1 Introduction

Faseela is an educational and interactive game designed to instill environmental values and sustainable practices in children. It introduces players to **Harith**, a 9-year-old boy with a growing curiosity about nature, who embarks on a learning journey guided by **Uncle Saleem**, a knowledgeable and experienced farmer. Although there is no family relationship between **Harith** and **Uncle Saleem**, the bond they form is based on mentorship and shared passion for protecting the environment. Through engaging, hands-on activities such as planting seeds, watering trees, and monitoring their growth, players gain practical insights into key environmental concepts including biodiversity conservation, climate action, and responsible resource management. By interacting with the virtual ecosystem, children experience the challenges and rewards of nurturing the natural environment. **Faseela** is purposefully developed to align with global sustainability initiatives, most notably the United Nations' Sustainable Development Goals (SDGs), particularly SDG 13 (Climate Action), SDG 15 (Life on Land), and SDG 12 (Responsible Consumption and Production). Furthermore, the game directly supports Saudi Arabia's Vision 2030 by fostering environmental awareness and promoting participation in national projects such as the Saudi Green Initiative. By combining educational content with immersive gameplay, **Faseela** aims to empower the next generation with the knowledge, skills, and attitudes needed to contribute positively to a greener and more sustainable future.

2 Executive Summary

Each stage involves simple tasks that mimic real life agricultural practices. As the players cultivate their virtual fields, players earn stars as rewards, celebrating their progress and dedication to nurturing their trees.

Scene 1 – Welcome and Introduction

The player opens the application and is presented with the main interface. After pressing the "Start" button, a dialogue begins between **Harith** and **Uncle Saleem**. **Uncle Saleem** introduces the mission: to plant and grow a tree. He explains the importance of starting with land preparation.

Scene 2 – Clearing the Land

The player begins the first task: clearing the land from rocks and debris. To do this, the player selects a fork (a farming tool) and uses it to remove each stone by clicking on them. Once all stones are removed, the player receives the first star as a reward.

Scene 3 – Preparing the Soil

A new conversation between **Harith** and **Uncle Saleem** explains the next step. **Uncle Saleem** talks about the importance of soil preparation and introduces the upcoming tasks: digging, planting seeds, and adding fertilizer.

Scene 4 – Planting the Seeds

The player uses a shovel to dig holes in the soil. After digging, the player plants seeds in the holes and then adds fertilizer. Once all steps are completed, the player earns the second star.

Scene 5 – The Importance of Watering

Harith and **Uncle Saleem** have another dialogue, where **Uncle Saleem** explains how essential water is for the tree to grow. He emphasizes that watering is just as important as planting.

Scene 6 – Watering the Tree

The player selects a watering can and uses it to water the soil. As the player waters, the tree gradually begins to grow. Once the process is complete, the player receives the third and final star, marking the successful completion of the mission.

Scene 7 – Congratulations

After completing the mission, a final dialogue begins. **Uncle Saleem** congratulates **Harith** on planting his first tree. The "Exit" button appears. When the player clicks it, the game ends.

This game blends storytelling, interactive gameplay, and agricultural education to create a fun and meaningful learning experience. It teaches children step-by-step planting skills while encouraging engagement, responsibility, and curiosity about nature.

3 Abstraction

The project **Faseela** is a computer graphics-based educational simulation that models the essential stages of tree planting and environmental care in an interactive and simplified format. It is specifically designed to cultivate environmental values and sustainable practices among children.

Features Included in the Simulation:

- **3D Objects and Environment:** The simulation features colorful 3D models of landscapes, farming tools (fork, shovel, seed bag, fertilizer, watering can), and dynamic stages of tree growth. All models are created using Blender and imported into Unity.
- **Scenario and Storyline:** A structured narrative follows the journey of **Harith**, a young boy, under the mentorship of **Uncle Saleem**. The story progresses through sequential scenes representing real agricultural activities such as land preparation, planting, and nurturing trees.
- **Interactive Gameplay Mechanics:** Children engage in hands-on tasks such as clearing rocks, digging soil, planting seeds, fertilizing, and watering trees, with gameplay designed to be intuitive and educational.
- **Dialogue System:** Informative, simple text-based dialogues between **Harith** and **Uncle Saleem** guide players throughout the experience, providing educational instructions and encouragement.
- **Sound Effects Integration:** The simulation incorporates natural sound effects (e.g., digging sounds, and water pouring) to create a more immersive experience.

- **Reward System:** Players earn stars as rewards after completing each major task, encouraging motivation, achievement, and progression.
- **Visual Growth Feedback:** The simulation visually represents tree growth responding to player actions, helping children understand the results of their environmental care efforts.
- **Cartoon-Style Graphics:** The visuals are rendered in a colorful, child-friendly, cartoonish style that enhances engagement and ensures the game remains accessible and appealing to a young audience.

Features Excluded from the Simulation:

- **Complex Environmental Variables:** Detailed aspects like soil composition, seasonal changes, and weather dynamics are excluded to maintain simplicity.
- **Detailed Biological Growth Simulation:** Tree growth is illustrated through simplified animation stages rather than detailed biological modeling.
- **Advanced Artificial Intelligence:** Character behavior follows pre-designed scripted dialogues without adaptive AI decision-making.
- **Multiplayer or Online Features:** The simulation focuses on a single-player experience, excluding online or cooperative multiplayer functionalities.
- **Large-Scale Ecosystem Modeling:** Broader ecological interactions and long-term climate modeling are outside the scope of this simulation.

By focusing on intuitive and engaging activities, **Faseela** provides children with a hands-on educational experience that fosters environmental responsibility and introduces sustainable practices in a fun and meaningful way, aligning with global sustainability goals and Saudi Arabia's Vision 2030.

4 Algorithms and Computational Implementation

4.1 Pseudocode and C# Scripts for Game Scripts and Systems

4.1.1 Star Reward System

Objective:

Provide visual and audio feedback to reward the player after completing each major task.

Process:

The **StarManager** system tracks the player's progress and grants a star when specific conditions are met, such as clearing land, planting seeds, or watering the tree.

Upon granting a star, the corresponding star icon is updated from an empty to a filled sprite, accompanied by a scaling animation and a star sound effect.

The system ensures that each star is granted only once to avoid duplication.

Pseudocode:

```
On Task Completion:
  If star not already granted:
    Change star icon to filled sprite
    Play star sound
    Animate star scale (expand and shrink)
    Increment granted star count
    Mark star as granted
```

C# Script:

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
using System.Collections;

Unity Script (1 asset reference) | 5 references
public class StarManager : MonoBehaviour
{
    public Image[] stars;
    public GameObject starCanvas;
    public Sprite emptyStarSprite;
    public Sprite fullStarSprite;
    public AudioSource starSound;
    public bool starGranted = false;
    private int currentStar = 0;
    private static bool exists = false;

    Unity Message | 0 references
    private void Start()
    {
        UnityEngine.SceneManagement.SceneManager.sceneLoaded += OnSceneLoaded;
    }
}
```


⊞ Unity Message | 0 references

```
private void Awake()
{
    if (exists)
    {
        Destroy(gameObject);
        return;
    }

    DontDestroyOnLoad(gameObject);

    if(starCanvas != null)
        DontDestroyOnLoad(starCanvas);
    exists = true;
}
```

⊞ Unity Message | 0 references

```
private void OnDisable()
{
    UnityEngine.SceneManagement.SceneManager.sceneLoaded -= OnSceneLoaded;
}
```

2 references

```
private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name == "Cleaning" || scene.name == "Planting" || scene.name == "Watering")
    {
        starCanvas.SetActive(true);
        starGranted = false;
    }
    else
    {
        starCanvas.SetActive(false);
    }
}
```

3 references

```
public void GrantStar()
{
    if (starGranted)
        return;

    if (currentStar < stars.Length)
    {
        stars[currentStar].sprite = fullStarSprite;
        StartCoroutine(AnimateStar(stars[currentStar]));

        if (starSound != null)
            starSound.Play();

        currentStar++;
    }

    starGranted = true;
}
```

1 reference

```
IEnumerator AnimateStar(Image star)
{
    Vector3 originalScale = star.rectTransform.localScale;
    Vector3 targetScale = originalScale * 1.6f;
    float durationUp = 0.3f;
    float durationDown = 0.5f;

    float t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime / durationUp;
        float scaleT = Mathf.Sin(t * Mathf.PI * 0.5f);
        star.rectTransform.localScale = Vector3.Lerp(originalScale, targetScale, scaleT);
        yield return null;
    }

    t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime / durationDown;
        float scaleT = 1f - Mathf.Cos(t * Mathf.PI * 0.5f);
        star.rectTransform.localScale = Vector3.Lerp(targetScale, originalScale, scaleT);
        yield return null;
    }

    star.rectTransform.localScale = originalScale;
}
```

4.1.2 Clearing the Land Scene

In this scene, the player uses a farm tool (fork) to engage with the world and eliminate obstructions (rocks).

Tool Activation

Objective:

Enable the player to choose the fork tool as an indicator of readiness to start land clearing.

Process:

When the player clicks on the fork tool, the tool expands slightly to offer instant visual feedback. At the same time, an audio sound effect is triggered to improve the user experience. When activated, the system turns the global flag **toolActivated** on, enabling the player to hit rocks and start the land-clearing process.

Pseudocode:

On Fork Tool Click:

- Scale the fork by a small amount
- Set toolActivated = true
- Play fork activation sound

C# Script:

```
using UnityEngine;
using UnityEngine.Audio;

public class SelectFork : MonoBehaviour
{
    private Vector3 originalScale;
    public float scaleMultiplier = 1.1f;
    public static bool toolActivated = false;
    public AudioSource forkSound;

    void Start()
    {
        originalScale = transform.localScale;
    }

    private void OnMouseDown()
    {
        transform.localScale = originalScale * scaleMultiplier;
        toolActivated = true;
        forkSound.Play();
    }

    public void ResetTool()
    {
        transform.localScale = originalScale;
        toolActivated = false;
    }
}
```

Object Removal

Objective:

Permit the player to clear all rocks from the land using the activated fork.

Process:

During gameplay, the system repeatedly waits for player mouse input. When the fork tool has been engaged, each left mouse click emits a ray from the camera through the clicked point. When the ray impacts a Removable-labeled object, the object is destroyed instantly, and the system increases a count of the number of removed objects. After clearing all eight compulsory rocks in which the player must succeed, a star is given through the **StarManager** system as a reward for the achievement.

Pseudocode:

During gameplay:

 If toolActivated:

 On mouse left button click:

 Cast a ray to mouse position

 If the ray collides with Removable object:

 Destroy the object

 Increment removedCount by 1

 If removedCount $\geq$ totalRemovables::

 Grant first star to player

[Load](#) next scene "ThirdCon" after 2 seconds

C# Script:

```
using UnityEngine;
using UnityEngine.SceneManagement;
using System.Collections;

public class ObjectRemover : MonoBehaviour
{
    private int totalRemovables = 8;
    private int removedCount = 0;

    void Update()
    {
        if (!SelectFork.toolActivated)
            return;

        if (Input.GetMouseButtonDown(0))
        {
            Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
            RaycastHit hit;

            if (Physics.Raycast(ray, out hit, 100f))
            {
                if (hit.collider.CompareTag("Removable"))
                {
                    Destroy(hit.collider.gameObject);
                    removedCount++;

                    if (removedCount >= totalRemovables)
                    {
                        FindObjectOfType<StarManager>().GrantStar();
                        StartCoroutine(LoadNextScene("ThirdCon"));
                    }
                }
            }
        }
    }

    IEnumerator LoadNextScene(string sceneName)
    {
        yield return new WaitForSeconds(2f);
        SceneManager.LoadScene(sceneName, LoadSceneMode.Single);
    }
}
```

Automatic Object Cleanup

Objective:

Ensure that any rocks which fall accidentally onto the ground surface are automatically removed.

Process:

During the play of the game, collision events are monitored so that the scene is kept clean. If a colliding object hits another object tagged as Ground, the system immediately removes the colliding object. This keeps the intended environment configuration and avoids needless mess.

Pseudocode:

On collision event:
the object collided with has the tag "Ground"
Destroy the object

C# Script:

```
using UnityEngine;

public class DestroyOnGround : MonoBehaviour
{
    void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
        {
            Destroy(gameObject);
        }
    }
}
```

4.1.3 Planting the Seeds Scene

In this scene, the player can interactively use farming tools and environmental elements one after the other to imitate planting.

Land Preparation

Objective:

Make the player till and dig the land to initiate the process of seeding.

Process:

Upon reaching the shovel tool, the shovel protrudes briefly for temporary visual feedback. Upon clicking, the system will swap the original ground object **groundNormalPrefab** with the new ground type variant depending on the clicks—"dug" or "after-dug". For the sake of immersion, a dig audio effect is activated, and the system advances an internal step counter to monitor the player's progress as they are ready to plant.

Pseudocode:

On Shovel Click:

- Enlarge shovel tool temporarily

- Remove existing ground object

- If clickCount == 0:

- Instantiate groundDugPrefab

- Else if clickCount == 1:

- Instantiate groundAfterDugPrefab

- Grant second star to player after 1 second

- Load next scene ("FourthCon") after 2.3 seconds

- Else:

- Reset ground to groundNormalPrefab

- Reset clickCount

- Increment clickCount

- Play dig sound

- After 0.5 seconds, reset tool scale

- Increment step

C# Script:

```
using UnityEngine;
using System.Collections;
using UnityEngine.SceneManagement;

public class ShovelTool : MonoBehaviour
{
    public static int step = 0;
    public GameObject groundNormalPrefab;
    public GameObject groundDugPrefab;
    public GameObject groundAfterDugPrefab;
    public AudioClip digSound;
    public float scaleMultiplier = 1.2f;
    private Vector3 originalScale;
    private GameObject currentGround;
    private int clickCount = 0;
    private Vector3 groundPosition = new Vector3(-6.291511f, -0.35f, -14.3645f);
    private AudioSource audioSource;
    private StarManager starManager;

    void Start()
    {
        originalScale = transform.localScale;

        audioSource = GetComponent();

        if (groundNormalPrefab == null)
        {
            return;
        }

        currentGround = Instantiate(groundNormalPrefab, groundPosition, Quaternion.identity);

        starManager = FindObjectOfType<StarManager>();
    }

    void ResetScale()
    {
        transform.localScale = originalScale;
    }

    IEnumerator DelayedGrantStar()
    {
        yield return new WaitForSeconds(1f);
        starManager.GrantStar();
    }

    IEnumerator LoadNextScene(string sceneName)
    {
        yield return new WaitForSeconds(2.3f);
        SceneManager.LoadScene(sceneName, LoadSceneMode.Single);
    }
}
```

```

private void OnMouseDown()
{
    if (step != 0 && step != 3) return;

    transform.localScale = originalScale * scaleMultiplier;

    Vector3 pos = groundPosition;
    Quaternion rot = Quaternion.identity;

    Destroy(currentGround);

    if (clickCount == 0)
    {
        currentGround = Instantiate(groundDugPrefab, pos, rot);
    }
    else if (clickCount == 1)
    {
        currentGround = Instantiate(groundAfterDugPrefab, pos, rot);

        if (starManager != null)
            StartCoroutine(DelayedGrantStar());

        StartCoroutine(LoadNextScene("FourthCon"));
    }
    else
    {
        currentGround = Instantiate(groundNormalPrefab, pos, rot);
        clickCount = -1;
    }

    clickCount++;

    if (digSound != null && audioSource != null)
    {
        audioSource.PlayOneShot(digSound);
    }

    Invoke(nameof(ResetScale), 0.5f);

    step++;
}

```

Seed Planting

Objective:

Once the ground has been dug, let the player plant seeds there.

Process:

After the dig step is complete and the internal step counter is 1, the player is able to interact with the seed bag. When clicked, it moves to the intended drop zone and rotates to mimic spilling seeds. Every seed is created one at a time at random positions on the drop zone, with randomized force applied for a natural-spilling effect. During this behavior, a sound effect of seed pouring plays for added immersion for the player. When every seed has been scattered, the step counter is incremented to advance to the next task and the seed bag returns to its starting position.

Pseudocode:

On Seed Bag Click:

If not busy:

 Start HandleSeedDropSequence coroutine

 Increment step

HandleSeedDropSequence:

 Set isBusy = true

 Move tool toward drop zone

 Rotate tool by 180 degrees

 Play seed sound

 For 20 times:

 Spawn a seed at a random slight offset

 Add random impulse to the seed

 Wait 0.1 seconds

 Stop seed sound

 Rotate tool back to original rotation

 Move tool back to original position

 Set isBusy = false

C# Script:

```
using UnityEngine;
using System.Collections;

public class SeedDropper : MonoBehaviour
{
    public Transform dropZoneTarget;
    public Transform spawnPoint;
    public GameObject seedPrefab;
    public int seedCount = 20;
    public float moveSpeed = 2f;
    public float rotationAmount = 180f;
    public float rotationSpeed = 90f;
    public AudioSource seedSound;

    private bool isBusy = false;
    private Vector3 originalPosition;
    private Quaternion originalRotation;

    void Start()
    {
        originalPosition = transform.position;
        originalRotation = transform.rotation;
    }

    void OnMouseDown()
    {
        if (ShovelTool.step != 1) return;

        if (!isBusy)
        {
            StartCoroutine(HandleSeedDropSequence());
            ShovelTool.step++;
        }
    }
}
```

```

IEnumerator HandleSeedDropSequence()
{
    isBusy = true;

    while (Vector3.Distance(transform.position, dropZoneTarget.position) > 0.01f)
    {
        transform.position = Vector3.MoveTowards(transform.position, dropZoneTarget.position, moveSpeed * Time.deltaTime);
        yield return null;
    }

    Quaternion startRot = transform.rotation;
    Quaternion targetRot = transform.rotation * Quaternion.Euler(0f, rotationAmount, 0f);

    float t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime * (rotationSpeed / 90f);
        transform.rotation = Quaternion.Slerp(startRot, targetRot, t);
        yield return null;
    }

    yield return new WaitForSeconds(0.5f);
    seedSound.Play();

    for (int i = 0; i < seedCount; i++)
    {
        Vector3 offset = new Vector3(Random.Range(-0.1f, 0.1f), 0, Random.Range(-0.1f, 0.1f));
        GameObject seed = Instantiate(seedPrefab, spawnPoint.position + offset, Quaternion.identity);

        Rigidbody rb = seed.GetComponent<Rigidbody>();
        if (rb != null)
        {
            Vector3 randomForce = new Vector3(Random.Range(-1f, 1f), -1f, Random.Range(-1f, 1f));
            rb.AddForce(randomForce * 2f, ForceMode.Impulse);
        }

        yield return new WaitForSeconds(0.1f);
    }

    seedSound.Stop();

    t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime * (rotationSpeed / 90f);
        transform.rotation = Quaternion.Slerp(transform.rotation, originalRotation, t);
        yield return null;
    }

    while (Vector3.Distance(transform.position, originalPosition) > 0.01f)
    {
        transform.position = Vector3.MoveTowards(transform.position, originalPosition, moveSpeed * Time.deltaTime);
        yield return null;
    }

    isBusy = false;
}
}

```

Fertilizer Spreading

Objective:

After planting the seeds, allow the player to distribute fertilizer.

Process:

The player can click on the fertilizer bag after the seeds have been successfully planted and the step counter has reached two. The fertilizer bag moves towards the drop zone and rotates to simulate spreading fertilizer. Fertilizer particles are spawned at random positions, and randomized impulse forces are given to them to simulate natural distribution. A pouring fertilizer sound effect is played during this. After all fertilizer is deposited, the bag returns to its starting point.

Pseudocode:

On Fertilizer Bag Click:

If not busy:

 Start HandleFertilizerDropSequence coroutine

 Increment step

HandleFertilizerDropSequence:

 Set isBusy = true

 Move tool toward drop zone

 Rotate tool by 180 degrees

 Play fertilizer sound

 For 20 times:

 Spawn a fertilizer at a random slight offset

 Add random impulse to the fertilizer

 Wait 0.1 seconds

 Stop fertilizer sound

 Rotate tool back to original rotation

 Move tool back to original position

 Set isBusy = false

C# Script:

```
using UnityEngine;
using System.Collections;

public class FR : MonoBehaviour
{
    public Transform dropZoneTarget;
    public Transform spawnPoint;
    public GameObject fertilizerPrefab;
    public int fertilizerCount = 20;
    public float moveSpeed = 2f;
    public float rotationAmount = 180f;
    public float rotationSpeed = 90f;
    public AudioSource fertilizerSound;
    private bool isBusy = false;
    private Vector3 originalPosition;
    private Quaternion originalRotation;

    void Start()
    {
        originalPosition = transform.position;
        originalRotation = transform.rotation;
    }

    void OnMouseDown()
    {
        if (ShovelTool.step != 2) return;

        if (!isBusy)
        {
            StartCoroutine(HandleFertilizerDropSequence());
            ShovelTool.step++;
        }
    }
}
```

```

IEnumerator HandleFertilizerDropSequence()
{
    isBusy = true;

    while (Vector3.Distance(transform.position, dropZoneTarget.position) > 0.01f)
    {
        transform.position = Vector3.MoveTowards(transform.position, dropZoneTarget.position, moveSpeed * Time.deltaTime);
        yield return null;
    }

    Quaternion startRot = transform.rotation;
    Quaternion targetRot = transform.rotation * Quaternion.Euler(0f, rotationAmount, 0f);

    float t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime * (rotationSpeed / 90f);
        transform.rotation = Quaternion.Slerp(startRot, targetRot, t);
        yield return null;
    }

    yield return new WaitForSeconds(0.5f);
    fertilizerSound.Play();

    for (int i = 0; i < fertilizerCount; i++)
    {
        Vector3 offset = new Vector3(Random.Range(-0.1f, 0.1f), 0, Random.Range(-0.1f, 0.1f));
        GameObject fertilizer = Instantiate(fertilizerPrefab, spawnPoint.position + offset, Quaternion.identity);

        Rigidbody rb = fertilizer.GetComponent<Rigidbody>();
        if (rb != null)
        {
            Vector3 randomForce = new Vector3(Random.Range(-1f, 1f), -1f, Random.Range(-1f, 1f));
            rb.AddForce(randomForce * 2f, ForceMode.Impulse);
        }

        yield return new WaitForSeconds(0.1f);
    }

    yield return new WaitForSeconds(0.5f);
    fertilizerSound.Stop();

    t = 0f;
    while (t < 1f)
    {
        t += Time.deltaTime * (rotationSpeed / 90f);
        transform.rotation = Quaternion.Slerp(transform.rotation, originalRotation, t);
        yield return null;
    }

    while (Vector3.Distance(transform.position, originalPosition) > 0.01f)
    {
        transform.position = Vector3.MoveTowards(transform.position, originalPosition, moveSpeed * Time.deltaTime);
        yield return null;
    }

    isBusy = false;
}
}

```


4.1.4 Watering the Tree Scene

In this scene, the player uses the watering can to irrigate the soil and trigger the growth of the tree. This interaction simulates real-life plant care by combining movement, animation, and timed effects. The process also rewards the player with a star upon successful completion.

Watering Action

Objective:

Allow the player to perform the watering action using the watering can, with simulated movement, rotation, particle effects, and audio.

Process:

When the player clicks on the watering can, it moves toward a predefined target position near the tree. The can then tilts to simulate pouring, while a water particle effect and watering sound are played. This interaction triggers the start of the tree growth sequence. After the watering is complete, the can returns to its original position, and the player is rewarded with a star. Finally, the scene transitions automatically to the next one.

Pseudocode:

On Watering Can Click:

- Move watering can to target position

- Tilt can to pouring angle

- Play water particles and sound

- Start GrowTree coroutine

- After watering:

 - Stop particles and sound

 - Return can to original position

 - Grant third star to player

 - Load scene “FifthCon” after 2 seconds

C# Script:

```
using UnityEngine;
using System.Collections;
using UnityEngine.SceneManagement;

Unity Script (1 asset reference) | 0 references
public class TeapotController : MonoBehaviour
{
    public Transform targetPoint;
    public float moveSpeed = 2f;
    public float rotateAmount = 30f;
    public float waitAfterGrowth = 1f;
    public TreeGrowthController treeGrowth;
    public ParticleSystem waterParticles;
    public AudioSource waterSound;
    private StarManager starManager;
    private Vector3 originalPosition;
    private Quaternion originalRotation;
    private bool isMoving = false;

    Unity Message | 0 references
    void Start()
    {
        originalPosition = transform.position;
        originalRotation = transform.rotation;
        starManager = FindObjectOfType<StarManager>();
    }

    Unity Message | 0 references
    void OnMouseDown()
    {
        if (!isMoving)
            StartCoroutine(MoveAndWaitForTree());
    }
}
```

1 reference

```
IEnumerator MoveAndWaitForTree()
{
    isMoving = true;

    Quaternion startRotation = transform.rotation;

    yield return MoveTo(targetPoint.position, transform.rotation);

    float tiltAmount = -30f;
    Quaternion tiltRotation = Quaternion.Euler(0f, 0f, tiltAmount) * startRotation;
    yield return RotateTo(tiltRotation);

    waterParticles.Play();
    yield return new WaitForSeconds(0.5f);
    waterSound.Play();

    yield return StartCoroutine(treeGrowth.GrowTree());

    waterParticles.Stop();
    yield return new WaitForSeconds(1.5f);
    waterSound.Stop();

    yield return RotateTo(startRotation);

    yield return MoveTo(originalPosition, originalRotation);

    if (starManager != null)
        starManager.GrantStar();

    StartCoroutine(LoadNextScene("FifthCon"));

    isMoving = false;
}
```

2 references

```
IEnumerator MoveTo(Vector3 targetPos, Quaternion targetRot)
{
    while (Vector3.Distance(transform.position, targetPos) > 0.01f)
    {
        transform.position = Vector3.MoveTowards(transform.position, targetPos, moveSpeed * Time.deltaTime);
        yield return null;
    }
}
```

2 references

```
IEnumerator RotateTo(Quaternion targetRot)
{
    while (Quaternion.Angle(transform.rotation, targetRot) > 0.5f)
    {
        transform.rotation = Quaternion.RotateTowards(transform.rotation, targetRot, 100 * Time.deltaTime);
        yield return null;
    }
}
```

1 reference

```
IEnumerator LoadNextScene(string sceneName)
{
    yield return new WaitForSeconds(2f);
    SceneManager.LoadScene(sceneName, LoadSceneMode.Single);
}
```

Tree Growth System

Objective:

Visually simulate the progressive growth of the tree through predefined growth stages after watering.

Process:

The tree consists of multiple GameObject stages representing its growth. At the beginning, all stages are hidden. Once watering is complete, the GrowTree() coroutine is triggered. This coroutine activates each stage in sequence, waiting for a set interval between stages, while deactivating the previous one to simulate natural growth.

Pseudocode:

At Start:

- Deactivate all growth stage objects

On GrowTree Coroutine:

- For each growth stage:

 - Deactivate previous stage (if any)

 - Activate current stage

 - Wait for growth interval

C# Script:

```
using UnityEngine;
using System.Collections;

Unity Script (1 asset reference) | 1 reference
public class TreeGrowthController : MonoBehaviour
{
    public GameObject[] growthStages;
    public float growthInterval = 3f;

    Unity Message | 0 references
    void Start()
    {
        foreach (GameObject stage in growthStages)
        {
            stage.SetActive(false);
        }
    }

    1 reference
    public IEnumerator GrowTree()
    {
        for (int i = 0; i < growthStages.Length; i++)
        {
            if (i > 0)
                growthStages[i - 1].SetActive(false);

            growthStages[i].SetActive(true);

            if (i < growthStages.Length - 1)
                yield return new WaitForSeconds(growthInterval);
        }
    }
}
```

4.2 Dialogue Code

```
Assets > C# DialogueManager.cs > DialogueManager > DisplayNextSentence
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5  using TMPro;
6
7  0 references
8  public class DialogueManager : MonoBehaviour
9  {
10     1 reference
11     public TMP_Text nameText;
12     2 references
13     public TMP_Text dialogueText;
14
15     5 references
16     private Queue<string> sentences;
17
18     0 references
19     void Start()
20     {
21         sentences = new Queue<string>();
22     }
23
24     public void StartDialogue(Dialogue dialogue)
25     {
26
27         nameText.text = dialogue.name;
28
29         sentences.Clear();
30
31         foreach (string sentence in dialogue.sentences)
32         {
33             sentences.Enqueue(sentence);
34         }
35
36         DisplayNextSentence();
37     }
38 }
```

```

1 reference
35 public void DisplayNextSentence()
36 {
37     if (sentences.Count == 0)
38     {
39         EndDialogue();
40         return;
41     }
42
43     string sentence = sentences.Dequeue();
44     StopAllCoroutines();
45     StartCoroutine(TypeSentence(sentence));
46 }
47
48
1 reference
49 IEnumerator TypeSentence(string sentence)
50 {
51     dialogueText.text = "";
52     foreach (char letter in sentence.ToCharArray())
53     {
54         dialogueText.text += letter;
55         yield return null;
56     }
57 }
58
1 reference
59 void EndDialogue()
60 {
61     Debug.Log("End of conversation.");
62 }
63 }

```

The **DialogueManager** script is responsible for displaying dialogues interactively at the beginning of the game. When the dialogue starts, it shows the speaker's name **Uncle Saleem** and displays the sentences one by one with a typing effect, letter by letter. It uses a queue to store the sentences in order and automatically ends the dialogue once all sentences have been shown. This script makes the introduction feel dynamic and engaging for the player.

```
Assets > C# DialogueTrigger.cs > DialogueTrigger
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 0 references
6 public class DialogueTrigger : MonoBehaviour
7 {
8     1 reference
9     public Dialogue dialogue;
10
11     0 references
12     public void TriggerDialogue()
13     {
14         FindObjectOfType<DialogueManager>().StartDialogue(dialogue);
15     }
16 }
```

The **DialogueTrigger** script is connected to buttons within the introduction scene. When the player presses a button such as Start or Continue, it triggers the **DialogueManager** to begin displaying the dialogue of **Uncle Saleem** introducing the player to the game's world and story.

```
Assets > C# Dialogue.cs > Dialogue
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 [System.Serializable]
6 0 references
7 public class Dialogue
8 {
9     0 references
10     public string name;
11
12     [TextArea(3, 25)]
13     0 references
14     public string[] sentences;
15 }
```

The **Dialogue** script acts as a simple data container that holds the speaker's name **Uncle Saleem** and the sentences that will be shown.


```
Assets > C# startCode.cs
1  using UnityEngine;
2  using UnityEngine.SceneManagement;
3
4  public class startCode : MonoBehaviour
5  {
6      public void LoadScene(string s)
7      {
8          SceneManager.LoadScene(s);
9      }
10
11     public void QuitGame()
12     {
13         Debug.Log("Game is quitting...");
14         Application.Quit();
15     }
16 }
```

The **startCode** script is used to control scene transitions and quit the game. It contains two main methods: **LoadScene()** which loads a new scene by name using `SceneManager.LoadScene()`. This method is linked to UI buttons like Start. The second method **QuitGame()** is used to exit the application. It first logs a message to the console for debugging purposes and then closes the game using `Application.Quit()`.

4.3 OpenGL Code Explanation

1. Setting Up the Environment:

- **GLCanvas:** A GLCanvas is created for OpenGL rendering and added to a Frame window.
- **Animator:** An Animator updates the scene at 60 FPS for smooth animation.
- **Window Closing:** A WindowListener is added to stop the animation and exit the program when the window closes.

2. Key Components:

- **Initialization (init):**
 - Set background color to gray (`gl.glClearColor`), enable depth testing (`gl.glEnable(GL.GL_DEPTH_TEST)`), and apply smooth shading (`gl.glShadeModel(GL.GL_SMOOTH)`).
 - Enable lighting and set up ambient, diffuse, and specular lighting.
 - Enable material properties for interaction with light.

3. Perspective and View Setup:

- **Reshape:** Adjust the viewport and apply a perspective projection (`glu.gluPerspective()`), defining near and far clipping planes.

4. Rendering the Tree Scene:

- **Camera Position:** Translate the camera to view the tree (`gl.glTranslatef`).
- **Rotation:** The scene rotates around the Y-axis with `gl.glRotatef(angle)`.

4.1 Tree Components:

- **Trunk (Cylinder):** Drawn using `glut.glutSolidCylinder()`, translated and rotated to be vertical with a dark brown color.
- **Branches (Small Cylinders):** Two branches drawn with `glut.glutSolidCylinder()`, placed symmetrically with a lighter brown color.
- **Leaves (Cones):** Three layers of cones are drawn with decreasing sizes and green color (`gl.glColor3f(0.0f, 0.6f, 0.0f)`).
- **Shadows:** A sphere is drawn to simulate a shadow with dark gray color.

5. Animation:

- The scene rotates continuously by incrementing the angle value.

4.3.1 OpenGL Result



4.3.2 OpenGL Code

```
package org.yourorgwhere;

import com.sun.opengl.util.Animator;
import com.sun.opengl.util.GLUT;
import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.media.opengl.GL;
import javax.media.opengl.GLAutoDrawable;
import javax.media.opengl.GLCanvas;
import javax.media.opengl.GLEventListener;
import javax.media.opengl.glu.GLU;

public class Tree_JOGL implements GLEventListener {

    private float angle = 0.0f;

    public static void main(String[] args) {

        // Create a window frame for the 3D scene
        Frame frame = new Frame("3D Tree JOGL");
        GLCanvas canvas = new GLCanvas();

        // Attach the OpenGL event listener
        canvas.addGLEventListener(new Tree_JOGL());
        frame.add(canvas);
        frame.setSize(640, 480);
        final Animator animator = new Animator(canvas);

        // Close the window gracefully
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                new Thread(new Runnable() {
                    public void run() {
                        animator.stop(); // Stop the animator thread
                        System.exit(0);   // Exit the program
                    }
                }).start();
            }
        });
    }
}
```

```

        frame.setLocationRelativeTo(null); // Center the window
        frame.setVisible(true);
        animator.start(); // Start the animation loop
    }

    public void init(GLAutoDrawable drawable) {

        GL gl = drawable.getGL();
        System.err.println("INIT GL IS: " + gl.getClass().getName());

        gl.setSwapInterval(1); // Set swap interval for smoother rendering

        // Set the background color (gray)
        gl.glClearColor(0.5f, 0.5f, 0.5f, 1.0f);

        gl.glEnable(GL.GL_DEPTH_TEST); // Enable depth testing for proper rendering
        gl.glShadeModel(GL.GL_SMOOTH); // Smooth shading for better appearance

        // Enable lighting
        gl.glEnable(GL.GL_LIGHTING);
        gl.glEnable(GL.GL_LIGHT0); // Enable the primary light source

        // Set up ambient light (soft lighting for the whole scene)
        float[] ambientLight = {0.2f, 0.2f, 0.2f, 1.0f};
        gl.glLightfv(GL.GL_LIGHT0, GL.GL_AMBIENT, ambientLight, 0);

        // Set up main light position and color (white light)
        float[] lightPos = {0.0f, 5.0f, 10.0f, 1.0f};
        float[] lightColor = {1.0f, 1.0f, 1.0f, 1.0f};
        gl.glLightfv(GL.GL_LIGHT0, GL.GL_POSITION, lightPos, 0);
        gl.glLightfv(GL.GL_LIGHT0, GL.GL_DIFFUSE, lightColor, 0);
        gl.glLightfv(GL.GL_LIGHT0, GL.GL_SPECULAR, lightColor, 0);

        // Set up specular highlights (shiny surfaces)
        float[] specularLight = {1.0f, 1.0f, 1.0f, 1.0f};
        gl.glLightfv(GL.GL_LIGHT0, GL.GL_SPECULAR, specularLight, 0);

        // Enable material properties for lighting
        gl.glEnable(GL.GL_COLOR_MATERIAL);
        gl.glColorMaterial(GL.GL_FRONT, GL.GL_AMBIENT_AND_DIFFUSE);
    }
}

```

```

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {

    GL gl = drawable.getGL();
    GLU glu = new GLU();

    // Prevent division by zero if the height is zero
    if (height <= 0) {
        height = 1;
    }

    // Set the viewport and aspect ratio based on the window size
    float h = (float) width / (float) height;
    gl.glViewport(0, 0, width, height);
    gl.glMatrixMode(GL.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(45.0f, h, 1.0, 50.0); // Perspective projection for 3D effect
    gl.glMatrixMode(GL.GL_MODELVIEW);
    gl.glLoadIdentity();
}

public void display(GLAutoDrawable drawable) {

    GL gl = drawable.getGL();
    GLUT glut = new GLUT();

    gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT); // Clear the screen and depth buffer
    gl.glLoadIdentity(); // Reset modelview matrix

    // Move the camera to get a better view of the tree
    gl.glTranslatef(0.0f, -3.0f, -18.0f); // Camera moved slightly forward

    // Apply rotation to the whole scene for dynamic effect
    gl.glRotatef(angle, 0.0f, 1.0f, 0.0f); // Rotate around the Y-axis
    angle += 0.5f; // Increment the rotation angle

    // === Drawing the trunk (brown cylinder) ===
    gl.glPushMatrix();
    gl.glColor3f(0.36f, 0.20f, 0.09f); // Dark brown color for the trunk
    gl.glTranslatef(0.0f, 2.0f, 0.0f); // Move the trunk up
    gl.glRotatef(-90.0f, 1.0f, 0.0f, 0.0f); // Rotate to make the cylinder vertical
    glut.glutSolidCylinder(0.5, 4.0, 20, 20); // Draw the trunk (radius, height, slices, stacks)
    gl.glPopMatrix();
}

```

```

// === Drawing branches (small cylinders) ===
gl.glColor3f(0.3f, 0.15f, 0.05f); // Lighter brown for the branches

// Draw the first branch
gl.glPushMatrix();
gl.glTranslatef(0.8f, 4.5f, 0.0f); // Move branch slightly to the left
gl.glRotatef(30.0f, 0.0f, 0.0f, 1.0f); // Rotate the branch
gl.glRotatef(-90.0f, 1.0f, 0.0f, 0.0f); // Vertical orientation
glut.glutSolidCylinder(0.2, 3.0, 10, 10); // Draw the first branch (radius, height)
gl.glPopMatrix();

// Draw the second branch
gl.glPushMatrix();
gl.glTranslatef(-0.8f, 4.5f, 0.0f); // Move branch to the right
gl.glRotatef(-30.0f, 0.0f, 0.0f, 1.0f); // Rotate the branch
gl.glRotatef(-90.0f, 1.0f, 0.0f, 0.0f); // Vertical orientation
glut.glutSolidCylinder(0.2, 3.0, 10, 10); // Draw the second branch (radius, height)
gl.glPopMatrix();

// === Drawing leaves (cone shapes) ===
gl.glColor3f(0.0f, 0.6f, 0.0f); // Green color for the leaves

float yOffset = 5.0f;
for (int i = 0; i < 3; i++) {
    gl.glPushMatrix();
    gl.glTranslatef(0.0f, yOffset, 0.0f); // Move each layer upwards
    gl.glRotatef(-90.0f, 1.0f, 0.0f, 0.0f); // Rotate for cone orientation
    glut.glutSolidCone(2.0f - i * 0.5f, 2.0f, 20, 20); // Draw cone with decreasing size upwards
    gl.glPopMatrix();
    yOffset += 1.2f; // Adjust position for the next layer
}

// === Adding shadows on the ground to create a 3D effect ===
gl.glPushMatrix();
gl.glTranslatef(0.0f, -10.0f, 0.0f); // Move the shadow down to the ground
gl.glColor3f(0.2f, 0.2f, 0.2f); // Dark gray color for the shadow
glut.glutSolidSphere(0.5f, 20, 20); // Draw a simple shadow using a sphere
gl.glPopMatrix();

gl.glFlush(); // Force the OpenGL pipeline to process the drawing
}

public void displayChanged(GLAutoDrawable drawable, boolean modeChanged, boolean deviceChanged) {
}
}

```

5 Results

5.1 3D Objects

Harith: The First Main Character



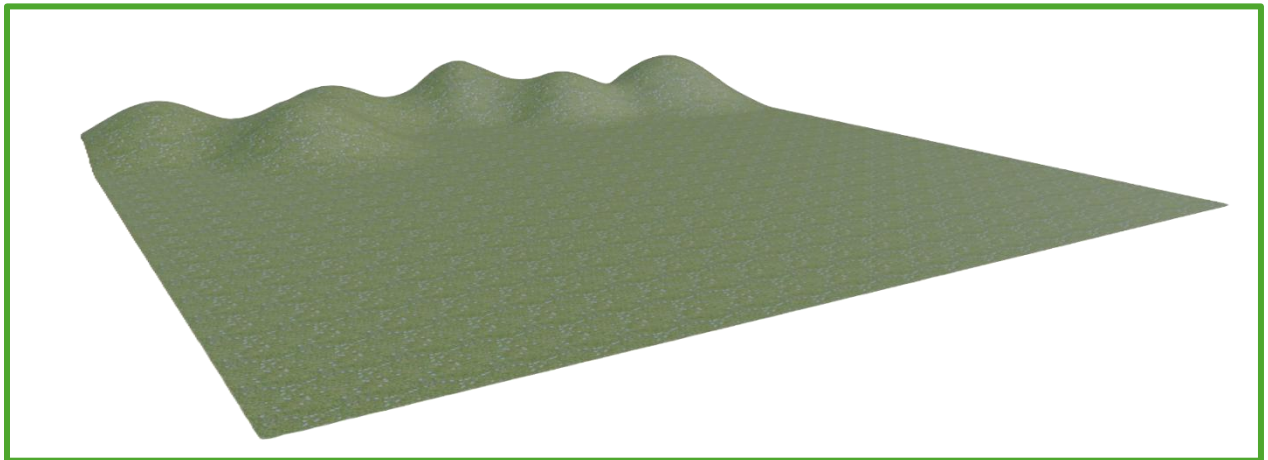
Uncle Saleem: The Second Main Character



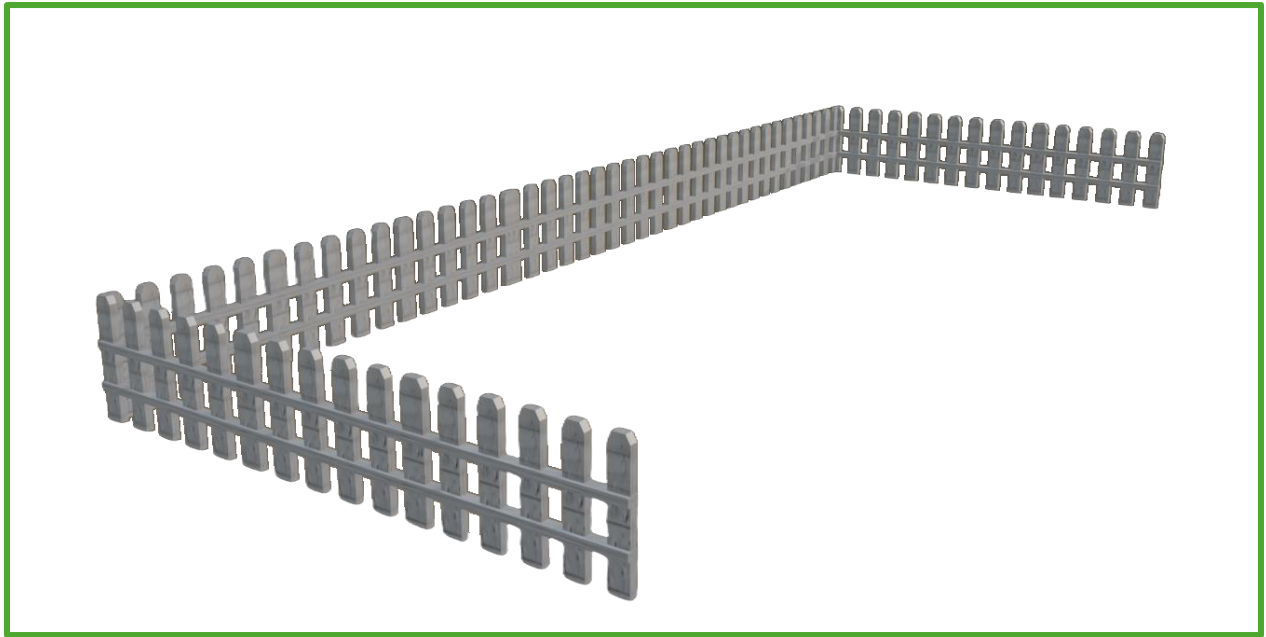
Soil and Rocks



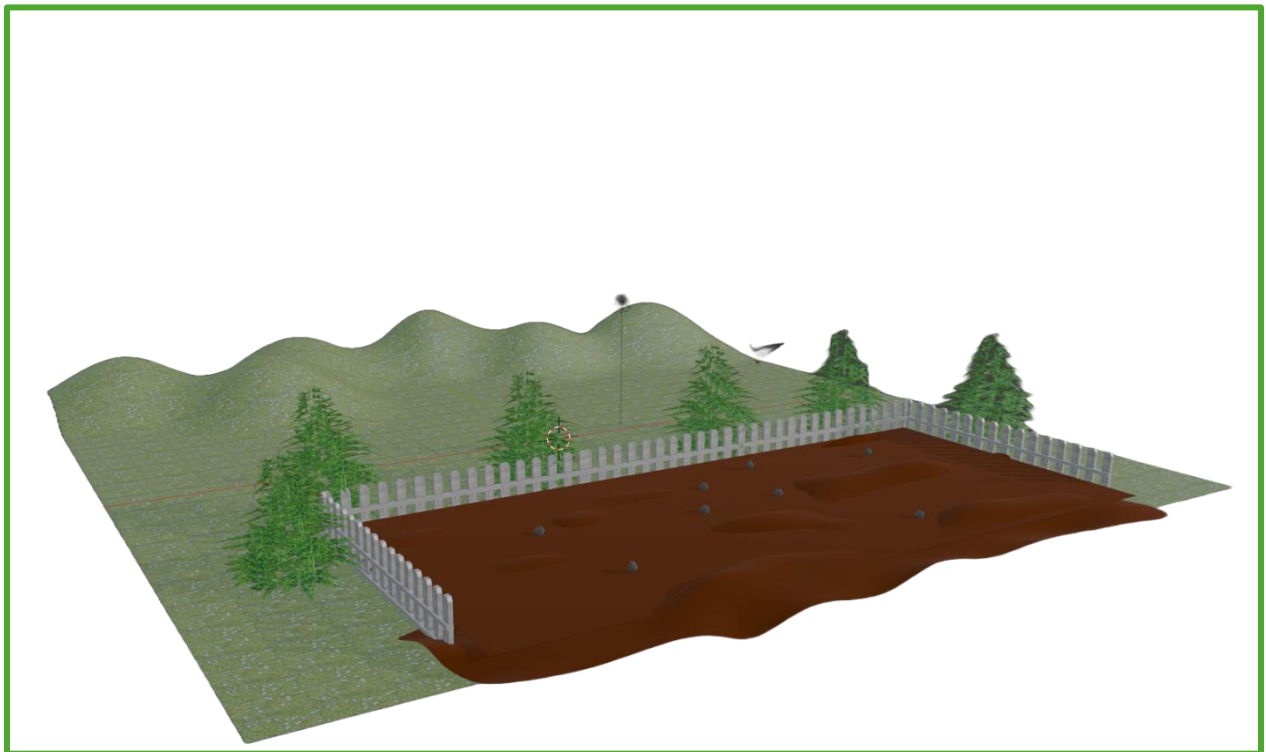
Ground Object



Fence Object



The Environment After Integrating the Objects



Tree Growth Stages



Watering Can



Shovel



Fork



5.2 Buttons

Start Button



Continue Buttons



Quit Button



5.3 Shots from the Game

The Start Screen



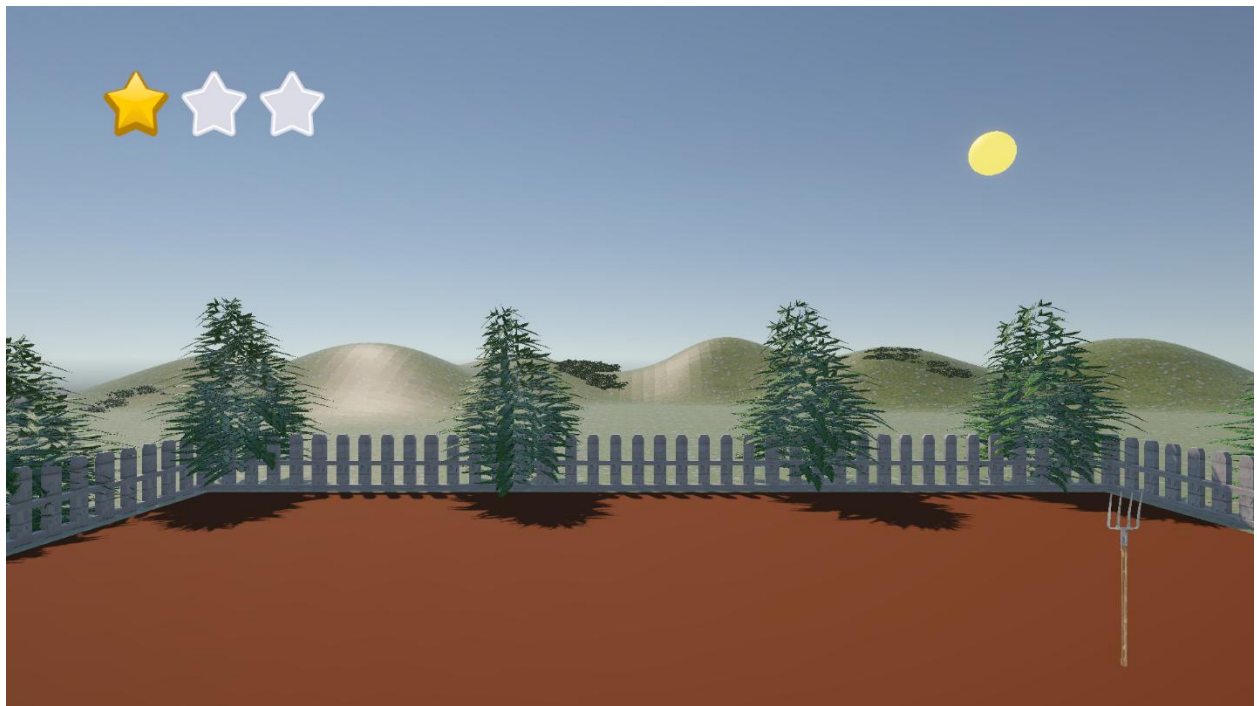
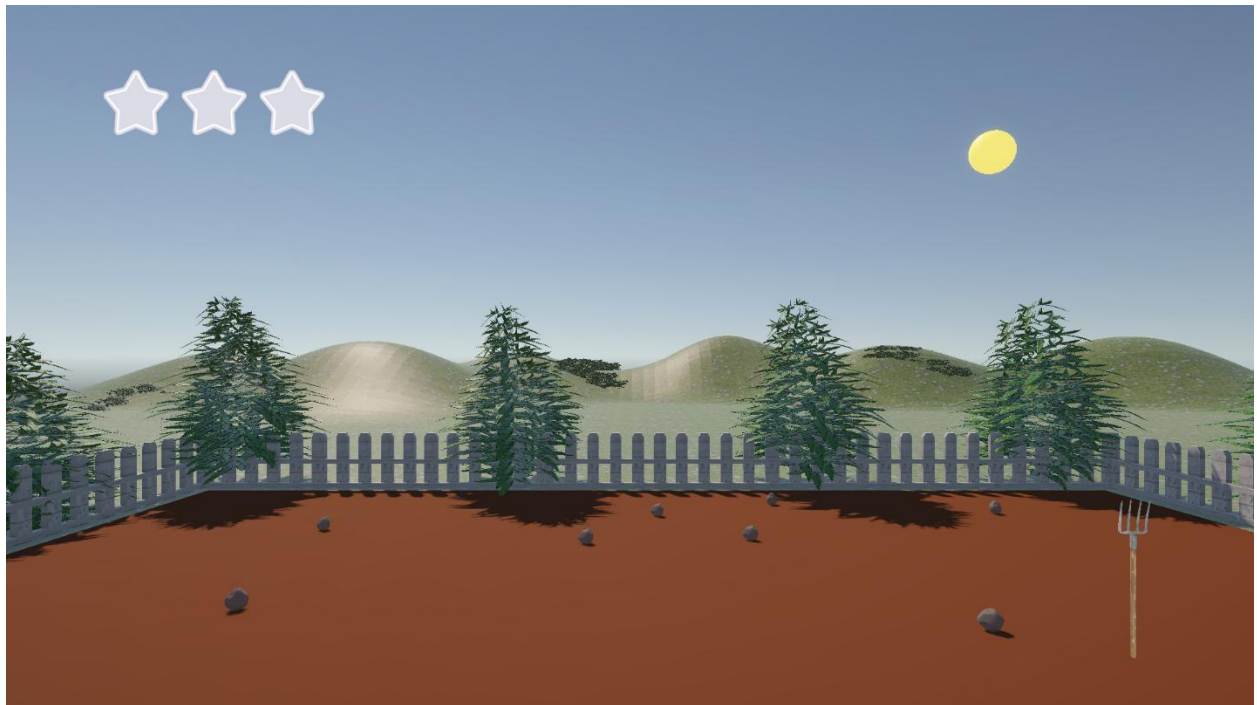
The First Conversation



The Second Conversation



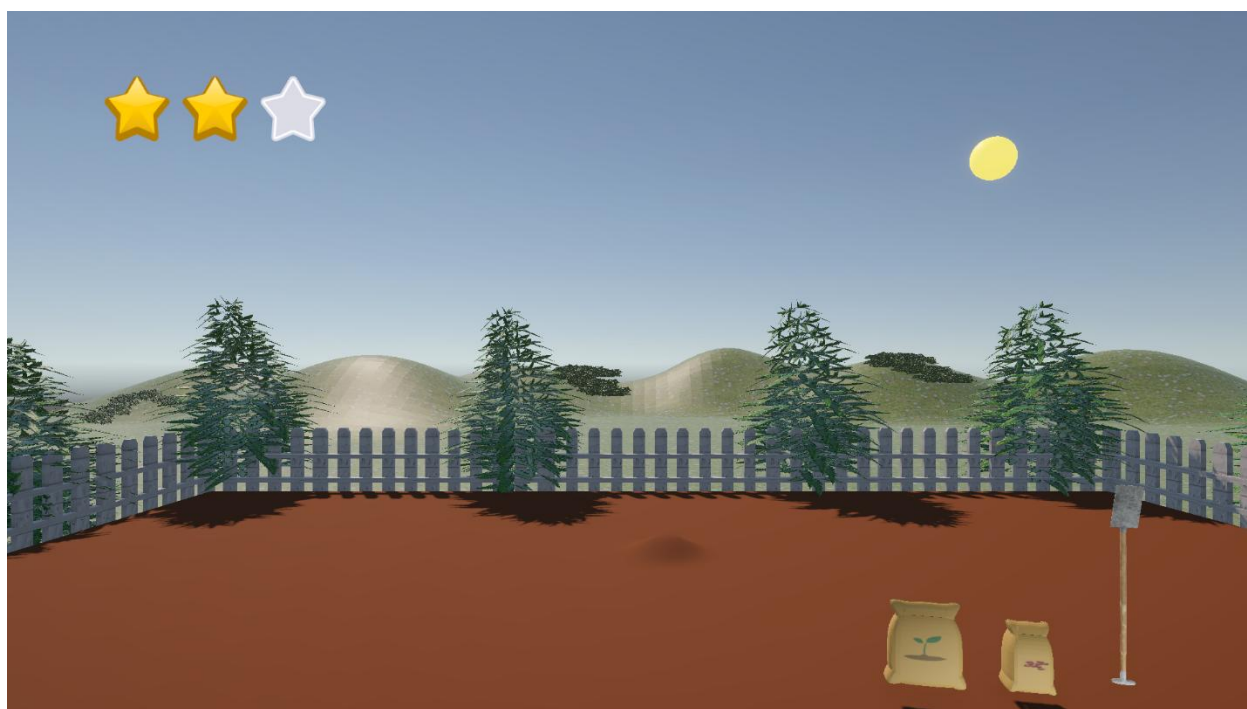
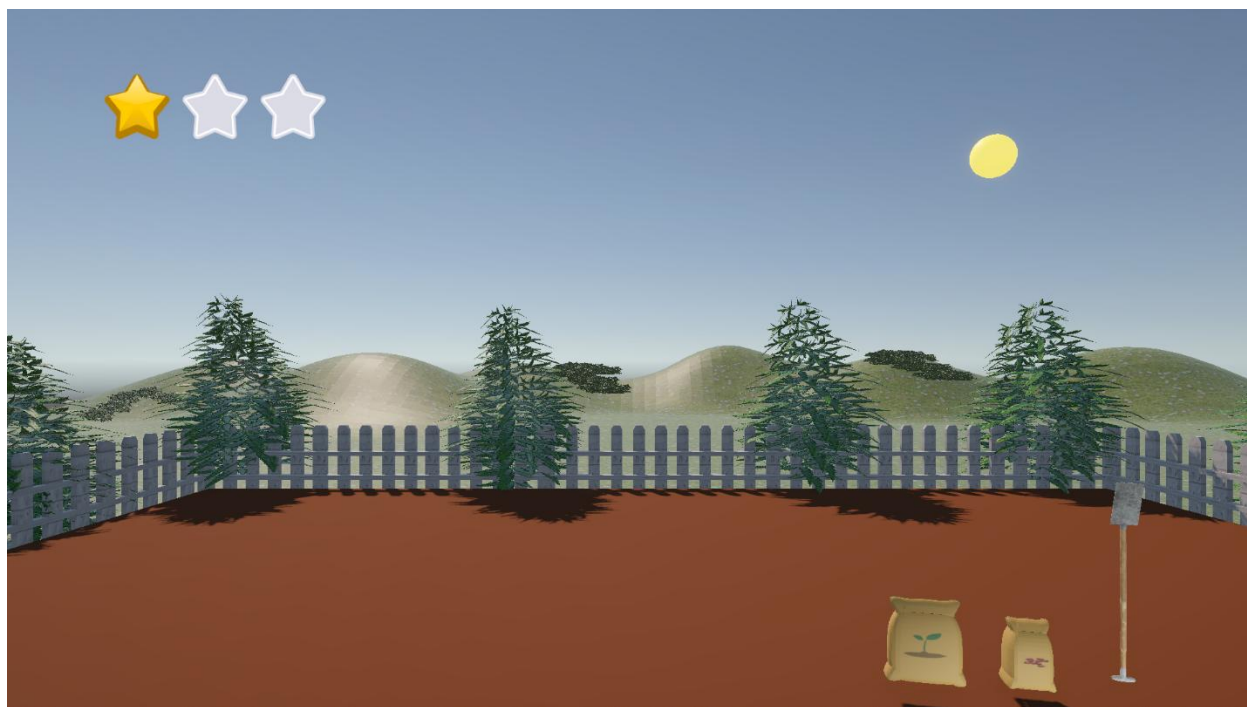
Clearing the Land Scene



The Third Conversation



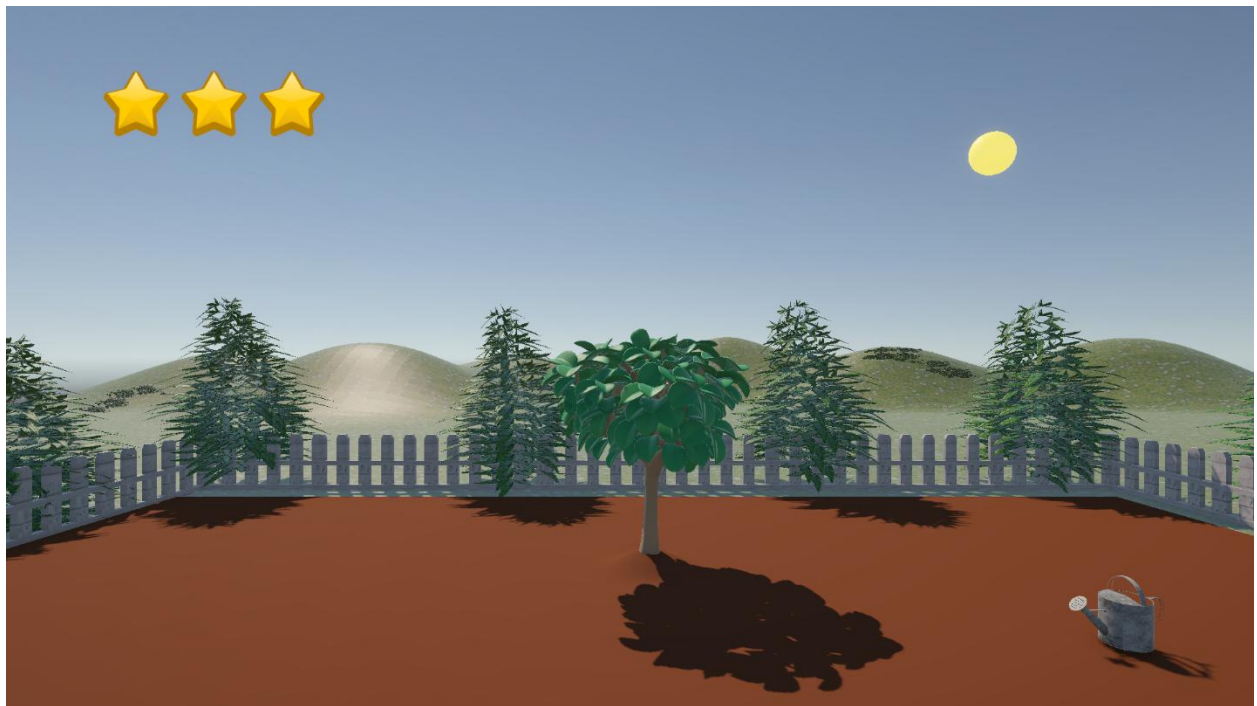
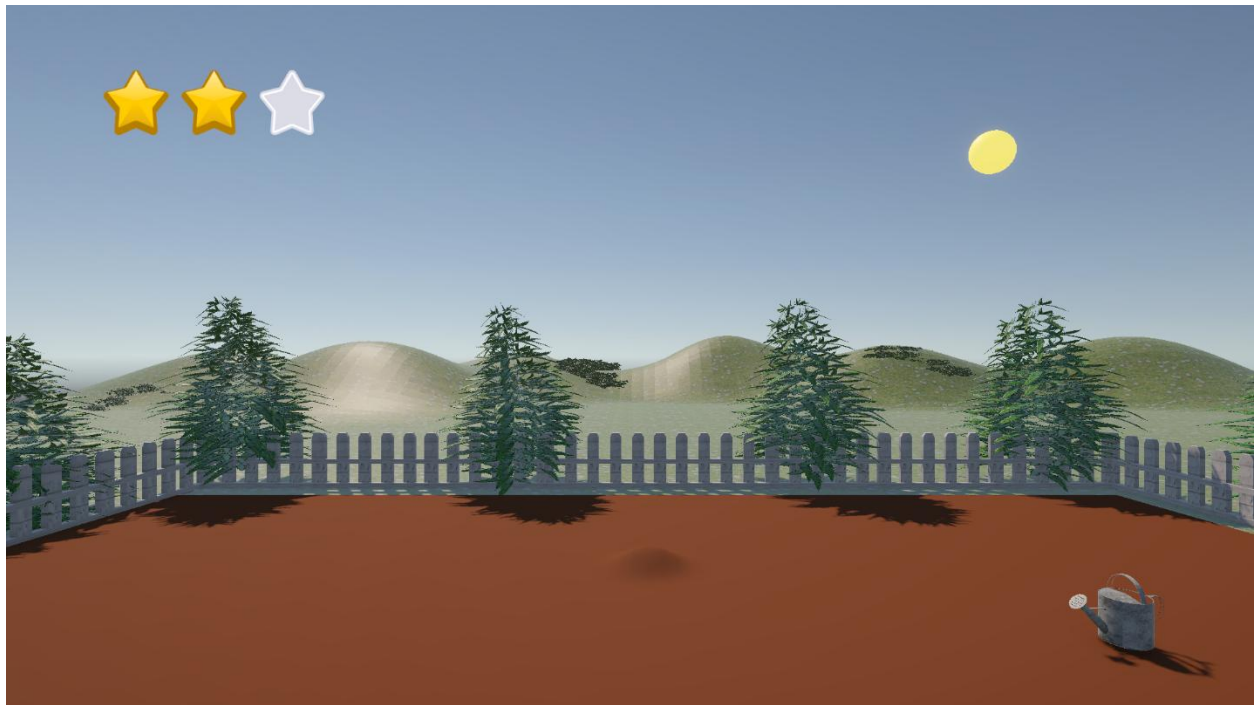
Planting the Seeds Scene



The Fourth Conversation



Watering the Tree Scene



The Fifth Conversation



The Final Conversation



6 Conclusion

6.1 Advantages of the Approach

Flexibility:

Using Unity and Blender provided great freedom in designing assets and scenes, enabling the implementation of diverse ideas and allowing for easy future expansion.

Visual Immersion:

High-quality 3D modeling helped create a visually engaging game environment, enhancing player interaction and enjoyment.

Character and Environment Design:

The main character, “Uncle Sellam,” and the nature-inspired local setting helped capture children’s attention and effectively deliver educational messages.

Enhanced User Experience:

The star-based reward system added motivation for children to complete tasks and stay engaged throughout the gameplay.

6.2 Disadvantages of the Approach

Resource Intensive:

High-quality 3D rendering requires powerful hardware to maintain optimal performance, which may limit compatibility with lower-end devices.

Development Time and Cost:

Creating detailed 3D models and textures is time-consuming, potentially extending the development timeline and increasing overall project costs.

6.3 Unexpected Aspects

Unity does not provide native support for Arabic text, resulting in reversed and disconnected characters. To resolve this issue, a pre-built script was integrated into the project to properly render and display Arabic text.

6.4 Future Work

In future updates, **Faseela** will introduce more complex challenges to provide a deeper and more engaging learning experience for players. One key addition will be the inclusion of varying weather conditions, such as rain, drought, and extreme temperatures, which will directly impact plant growth and survival. Players will need to adapt their strategies and manage these unpredictable environmental factors, learning about the importance of climate awareness and resilience in the face of changing weather patterns.

Additionally, **Faseela** will feature a wider variety of plants and soil types, each requiring different care techniques. This will challenge players to understand how different species thrive in different conditions, promoting a more nuanced understanding of biodiversity and the need for responsible resource management. For example, players may need to adjust their watering schedules based on soil composition or adapt their planting methods depending on the plant type.

These new features will help create a more dynamic and educational experience, allowing players to explore the real-world complexities of environmental care. By incorporating these challenges, **Faseela** will continue to foster important values related to sustainability, encouraging children to think critically about the environment and how their actions can support a more sustainable future.